

Many-to-Many RNNs for Machine Translation

From Encoder–Decoder to Attention

Already covered:

- Many-to-one RNNs for sequence classification
- Backpropagation through time (BPTT)

Now:

- Many-to-many sequence modeling for machine translation
- Vanilla encoder-decoder RNN
- Why a fixed context vector is a bottleneck
- Attention as a learned soft alignment mechanism
- Why RNN-based translation still has limitations, even with attention

Main pedagogical goal:

classification $\longrightarrow$ sequence generation $\longrightarrow$ alignment $\longrightarrow$ modern alternatives

Many-to-many setting in machine translation

Machine translation maps an input sequence in one language to an output sequence in another language.

$$x_1, x_2, \dots, x_{T_x} \longrightarrow y_1, y_2, \dots, y_{T_y}$$

Examples:

- English $\rightarrow$ French
- Tamil $\rightarrow$ English
- Sentence lengths need not match:

$$T_x \neq T_y$$

Key difference from many-to-one:

- In sentiment classification, one output label summarizes the whole sequence.
- In translation, the model must produce a *sequence* of outputs, one token at a time.

Probabilistic formulation of translation

The goal is to model the conditional probability of the target sentence given the source sentence:

$$P(y_1, \dots, y_{T_y} \mid x_1, \dots, x_{T_x})$$

Using the chain rule:

$$P(y_1, \dots, y_{T_y} \mid x) = \prod_{t=1}^{T_y} P(y_t \mid y_1, \dots, y_{t-1}, x)$$

So at each decoder step t , the model predicts:

$$P(y_t \mid y_{<t}, x)$$

Interpretation:

- The next translated word depends on:
 - the source sentence x
 - the target words already generated
- Translation is autoregressive generation

Vanilla seq2seq RNN: high-level idea

The classic RNN translation architecture has two parts:

Encoder:

- Reads the source sentence token by token
- Produces hidden states
- Final hidden state is used as a summary of the whole source

Decoder:

- Starts from this summary
- Generates target tokens one at a time
- Uses its own recurrent hidden state

Core idea:

source sentence $\longrightarrow$ single vector $\longrightarrow$ target sentence

This is elegant, but the single-vector summary becomes the central weakness.

Encoder equations

Let the source tokens be $x_1, \dots, x_{T_x}$.

Each token is embedded:

$$e_t = E_x(x_t)$$

The encoder RNN updates:

$$h_t = f_{\text{enc}}(e_t, h_{t-1})$$

where:

- e_t = embedding of source token x_t
- h_t = encoder hidden state at time t
- f_{enc} can be a vanilla RNN, GRU, or LSTM cell

After the full source sentence is read, the final state is:

$$c = h_{T_x}$$

Interpretation:

- c is the *context vector*
- It is supposed to summarize the entire source sentence

Decoder equations

The decoder generates one target word at a time.

Let the decoder hidden state at time t be s_t .

Initialization:

$$s_0 = \phi(c)$$

where c is the encoder context vector.

At each decoder step:

$$s_t = f_{\text{dec}}(E_y(y_{t-1}), s_{t-1})$$

Then logits are computed:

$$o_t = W_o s_t + b_o$$

and the next-token distribution is:

$$P(y_t \mid y_{<t}, x) = \text{softmax}(o_t)$$

Special token:

- Decoding begins from a <BOS> token
- Generation stops when <EOS> is produced

Why this is many-to-many

This is a many-to-many architecture because:

- The input is a sequence:

$$x_1, \dots, x_{T_x}$$

- The output is also a sequence:

$$y_1, \dots, y_{T_y}$$

But unlike sequence labeling, translation is not usually synchronous:

- input and output lengths differ
- word order changes
- one source word may correspond to multiple target words
- some target words may depend on long-range source context

So translation is a much harder many-to-many problem than tagging.

Training the seq2seq model

Training usually uses **teacher forcing**.

At decoder step t , instead of feeding the model's own previous prediction, we feed the true previous target token:

$$s_t = f_{\text{dec}}(E_y(y_{t-1}^{\text{true}}), s_{t-1})$$

The loss is the sum of token-level cross-entropies:

$$\mathcal{L} = - \sum_{t=1}^{T_y} \log P(y_t^{\text{true}} | y_{<t}^{\text{true}}, x)$$

Important point:

- The gradient must flow:
 - backward through decoder time steps
 - through the context vector c
 - backward through encoder time steps
- So training requires BPTT across a long computational graph

Why the fixed context vector is a bottleneck

In vanilla seq2seq:

$$c = h_{T_x}$$

must contain all information needed for the entire translation.

That means one vector must encode:

- content of all source words
- important long-range dependencies
- ordering information
- syntactic structure
- semantic relations needed later in decoding

For short sentences this may be manageable.

For long or complex sentences, it becomes unrealistic:

many words $\nrightarrow$ perfectly captured in one fixed-size vector

This is the **information bottleneck**.

Bottleneck...

Consider translating:

The book that the professor who visited the lab yesterday recommended was finally published.

A single context vector must preserve:

- the main subject
- subordinate clauses
- who did what
- tense and agreement information
- content needed many decoding steps later

The decoder may need specific source details at different times:

- early target tokens may depend on the source beginning
- later target tokens may depend on a source word in the middle

A single summary vector does not allow selective retrieval of different source positions.

It is useful to connect this to BPTT, but carefully.

In vanilla seq2seq, the loss at decoder step t influences encoder state h_k through a long path:

$$y_t \rightarrow s_t \rightarrow s_{t-1} \rightarrow \cdots \rightarrow s_0 \rightarrow c = h_{T_x} \rightarrow h_{T_x-1} \rightarrow \cdots \rightarrow h_k$$

So for early source words, gradients may have to traverse:

- many decoder steps
- the context bottleneck
- many encoder steps

Consequences:

- vanishing gradients make learning long-range dependencies hard
- hidden states are forced to preserve information over long spans
- the final encoder state is overloaded

So attention is motivated mainly by the bottleneck, and it also helps reduce this long-range optimization burden.

Key idea of attention

Instead of using only the final encoder state h_{T_x} , let the decoder look at *all* encoder states:

$$h_1, h_2, \dots, h_{T_x}$$

At decoder step t , compute a step-specific context vector:

$$c_t = \sum_{j=1}^{T_x} \alpha_{t,j} h_j$$

where:

- $\alpha_{t,j} \geq 0$
- $\sum_{j=1}^{T_x} \alpha_{t,j} = 1$

Interpretation:

- $\alpha_{t,j}$ tells how much decoder step t attends to source position j
- The decoder can focus on different source words at different times

Attention as soft alignment

In translation, there is often an implicit alignment between target and source words.

Example:

I eat apples $\longrightarrow$ Je mange des pommes

When generating *mange*, the decoder should focus on *eat*. When generating *pommes*, it should focus on *apples*.

Attention learns this softly:

- not one hard source position
- but a distribution over source positions

$$\alpha_{t,1}, \alpha_{t,2}, \dots, \alpha_{t,T_x}$$

So attention is a **soft alignment mechanism**.

General attention computation

At decoder step t , use the previous decoder state s_{t-1} and each encoder state h_j to compute a score:

$$e_{t,j} = a(s_{t-1}, h_j)$$

Then normalize using softmax:

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^{T_x} \exp(e_{t,k})}$$

Finally compute the context vector:

$$c_t = \sum_{j=1}^{T_x} \alpha_{t,j} h_j$$

Then the decoder state uses both the previous target token and this context:

$$s_t = f_{\text{dec}}(E_y(y_{t-1}), s_{t-1}, c_t)$$

Now the source representation is *dynamic* rather than fixed.

Typical score functions for attention

The function $a(s_{t-1}, h_j)$ can take different forms.

Dot product:

$$e_{t,j} = s_{t-1}^\top h_j$$

Bilinear / general form:

$$e_{t,j} = s_{t-1}^\top W_a h_j$$

Additive attention (Bahdanau-style):

$$e_{t,j} = v_a^\top \tanh(W_s s_{t-1} + W_h h_j + b_a)$$

Why additive attention became influential:

- flexible learned compatibility function
- works even when encoder and decoder spaces differ

Decoder with attention

Once c_t is computed, the decoder updates using source information specific to time step t .
A generic formulation is:

$$s_t = f_{\text{dec}}(E_y(y_{t-1}), s_{t-1}, c_t)$$

Then output logits may be formed using both the decoder state and the context:

$$o_t = W_o[s_t; c_t] + b_o$$

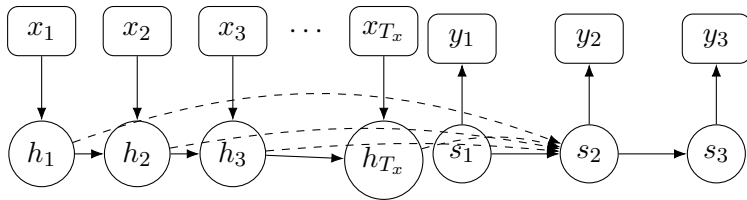
and the next-token distribution is:

$$P(y_t \mid y_{<t}, x) = \text{softmax}(o_t)$$

Conceptually:

- s_{t-1} says: what have I generated so far?
- c_t says: which source information matters now?

Diagram: seq2seq with attention



attention over all encoder states

At each decoder step, the model can consult all source positions instead of relying on one fixed summary.

Why attention helps

Attention helps in several ways.

1. Removes the fixed-length bottleneck

- The source is no longer compressed into only one vector.

2. Gives step-specific access to the source

- Different target words can look at different source words.

3. Improves gradient flow

- Decoder losses connect more directly to encoder states.
- The model need not preserve everything only in h_{T_x} .

4. Provides interpretability

- Attention weights often show approximate source-target alignments.

So attention is both an architectural improvement and an optimization aid.

How attention shortens the information path

Without attention, a decoder output may influence an early encoder state only through:

$$y_t \rightarrow s_t \rightarrow \cdots \rightarrow s_0 \rightarrow h_{T_x} \rightarrow \cdots \rightarrow h_k$$

With attention, decoder step t uses h_k more directly through:

$$h_k \rightarrow e_{t,k} \rightarrow \alpha_{t,k} \rightarrow c_t \rightarrow s_t \rightarrow y_t$$

Important nuance:

- The encoder is still recurrent.
- BPTT still exists.
- But the model no longer relies exclusively on the final encoder state.

This is why attention often works much better on long sentences.

Example of attention during translation

Suppose the source is:

$$x = (\text{the, red, car})$$

and target is:

$$y = (\text{la, voiture, rouge})$$

At different decoder steps:

- To produce *la*, attention may focus broadly
- To produce *voiture*, attention may peak on *car*
- To produce *rouge*, attention may peak on *red*

So the decoder is not asking:

“What was the whole sentence summary again?”

It is asking:

“Which parts of the source sentence matter right now?”

Teacher forcing and inference

Training:

- Use ground-truth previous token y_{t-1}^{true}
- Easier optimization

Inference:

- Previous token is the model's own prediction
- Errors can accumulate over time

At inference:

$$\hat{y}_t = \arg \max_y P(y \mid \hat{y}_{<t}, x)$$

or use beam search.

A practical issue:

- During training, decoder sees true history
- During inference, decoder sees its own imperfect history

This affects both vanilla and attention-based RNN translation.

Does attention solve all problems? No.

Attention is a major improvement, but RNN-based translation still has important limitations.

Even with attention:

- the encoder is still sequential
- the decoder is still sequential
- long-range dependencies are still harder than in fully non-recurrent models
- training remains difficult for very long sequences
- inference remains slow because tokens are generated one by one

So attention improves seq2seq RNNs a lot, but does not remove the fundamental sequential nature of RNNs.

Transformer based MT

Transformers addressed several limitations.

Key differences:

- no recurrent hidden-state chain across time
- self-attention connects tokens more directly
- much better parallelization during training
- easier modeling of long-range dependencies

Very roughly:

Vanilla RNN MT $\longrightarrow$ Attention-based RNN MT $\longrightarrow$ Transformer MT

So attention started as an improvement *inside* RNN seq2seq, and later became the central mechanism of a new architecture family.

Summary

Vanilla encoder–decoder RNN

- encodes source into one vector c
- decoder generates target from c
- suffers from a fixed-context bottleneck

Attention-based seq2seq

- decoder attends to all encoder states
- context becomes dynamic: c_t
- improves alignment, learning, and long-sentence handling

But even with attention

- recurrence remains sequential
- training and inference remain slower
- long-range modeling is still imperfect

This motivates the transition to Transformers.

Comparison table

	Vanilla Seq2Seq RNN	Seq2Seq RNN + Attention
Source representation	Single context vector c	All encoder states $\{h_j\}$
Decoder access to source	Indirect, fixed	Dynamic, step-specific
Long sentence handling	Weak	Better
Alignment behavior	Implicit only	Soft alignment via $\alpha_{t,j}$
Optimization	Hard over long paths	Improved, but still recurrent
Parallelism	Poor	Poor to moderate
Still limited by recurrence?	Yes	Yes